9a) Surcharge d'opérateur

DinoVec2 operator+ (DinoVec2 a, DinoVec2 b)

DinoVec2 operator* (DinoVec2 a, float b)

operator* (float a, DinoVec2 b)

operator -

... etc ...

a + b ⟺ operator+ (a, b)

a * b ⟺ operator* (a, b)

→ Priorité non-changeable

a + b × c + d ⟺ operator+ (operator+ (a, operator* (b, c)), d)

Alternative

```
struct DinoVec2 {
    DinoVec2 operator+(DinoVec2 other);
    DinoVec2 operator*(float other);
}
```

vec2 + vec2 $\Longleftrightarrow$ vec2.operator+(vec2)

vec2 × float $\Longleftrightarrow$ vec2.operator*(float)

~~float × vec2 $\Longleftrightarrow$ float operator*(vec2)~~

Float n'a pas de méthodes, donc pas possible

Ce qui est écrit :

```
struct DinoVertex {
    DinoVec2 pos{};
    uint16_t u = 0;
    uint16_t v = 0;
    DinoColor color = DinoColor_WHITE;
};
```

→ Constructeur implicitement généré

Ce que le compilateur génère :

```
struct DinoVertex {
    DinoVec2 pos;
    uint16_t u;
    uint16_t v;
    DinoColor color;

    DinoVertex(); // CONSTRUCTEUR
};
```

→ Constructeur sans argument → Constructeur par défaut

```
DinoVertex::DinoVertex()
{
    pos = {};
    u = 0;
    v = 0;
    color = DinoColor_WHITE;
}
```

→ Avantage du constructeur : On ne peut pas oublier de l'appeler !

```
DinoVertex v; // Le compilateur appelle implicitement le constructeur
```

```
std::vector<DinoVertex> vs;
vs.resize(6);
```

↳ std::vector::resize() appelle le constructeur par défaut sur chaque nouvel élément

9c]

SYNTAXE DU CONSTRUCTEUR

```
class DinoPlayer {
public:
    DinoPlayer(———);
};
```

N'importe quels arguments
(si pas d'argument, "constructeur par défaut")

Le constructeur est une méthode avec le nom du type

Le constructeur n'a pas de type de retour

```
DinoPlayer::DinoPlayer(———){
———
}
```

g_Players.resize(4) → Ne compile plus car pas de constructeur par défaut

g_Players.emplace_back(_____)

↳ Arguments transférés au constructeur de l'élément

9d) En symétrie du constructeur, en C++, quand une variable est détruite (quand elle n'est plus accessible) le compilateur appelle automatiquement le destructeur

SYNTAXE DU DESTRUCTEUR

```
class DinoPlayer {
 public :
        ~DinoPlayer();
};
```

Aucun argument

Alt Gr + 2

Le destructeur s'appelle "~" + le nom de la classe

Pas de type de retour

```
DinoPlayer :: ~DinoPlayer() {

}
```